

On this page

Installing Feedzai with Ansible

The Feedzai Ansible Toolkit refers to a series of Ansible Playbooks created by Feedzai to automate the services installation, and **manage configurations** across multiple environments.

In order to use it, a server (or container) must be provisioned for the purpose of having an [Ansible control node](#). This node must contain all the necessary binaries and dependencies for installing each component. Ansible needs to have the ability to connect to each server where the Feedzai platform will be installed, which means that all servers must be able to resolve the IP address (or hostnames) of each other, and the control node must be able to connect to them via SSH. For more information about the connectivity requirements check the [Ansible documentation](#).



The Ansible Playbooks provided by Feedzai can be used solely for the purpose of **configuration management**, and are not a solution for deploying Feedzai's products.

Requirements

- All servers hostnames must resolve their own IP's.
- The servers should be able to resolve each others hostnames, please update the `/etc/hosts` file if necessary, or use a DNS service.
 - Use `ping` to ensure to confirm that each server is able to ping other servers.
- A Linux-based Control Node (preferably dedicated to this task), with a deployment user configured. The node must have `Python3` for running Ansible.
- A deployment user on each server with superuser (`sudo`) permissions (some deployment steps include installing system services, and changing ownership of folders).
- SSH public key authentication between the deployment user in the control node and the deployment user in all servers.

Services Installation

To install a `<service>` (i.e.: `pulse` , `case-manager` , `reference-data`) with default configurations:

```
python3.6 ansible-playbook \
--connection=local \
-i project-default \
-i project-hsbc \
-i gcp \
<service>.yaml \
--tags=<service>.install
```

Copy

Services Configuration



The following steps are currently under review and assume we can run ansible locally to configure a `<services>` for a given environment `<env>` .

To configure a `<service>` for a given `<region>` (`uk` , `hk` , `asp`), portfolio (`payments` , `cards` , `amh` , `hase`) and `<env>` (e.g.: `dev` , `qa` , `prod` ...):

```
python3.6 ansible-playbook \
--connection=local \
-i project-default \
-i project-hsbc \
-i project-<env> \
-i <region>/project-<env> \
-i <region>/project-<env>/<portfolio> \
<service>.yaml \
--tags=<service>.configure
```

Copy

Notes

The `ansible_user` property needs to exist in the machine that will execute the ansible roles prior to the execution, by default, the `ansible_user` is `feedzai`. This applies to all playbook executions.

In the above command-line invocation of `ansible-playbook`, each corresponding inventories need to exist and contain a `variables.yml` file. If the same variable is defined in multiple inventories, the later ones appearing in the command-line invocation take precedence over the earlier ones. In this way, by adding the very specific variables at the end (for example, a region-specific, portfolio-specific, environment-specific set of variables), they can overwrite any values inherited from earlier inventories such as the default ones (e.g. `project-default`, ...).

Inventory	Directory that must exist with variables.yml
<code>project-default</code>	<code>project-default/group_vars/<service>/variables.yml</code>
<code>project-hsbc</code>	<code>project-hsbc/group_vars/<service>/variables.yml</code>
<code><region>/project-<env>/<portfolio></code>	<code><region>/project-<env>/<portfolio>/group_vars/<service>/variables.yml</code>
<code><region>/project-<env>/<portfolio></code>	<code><region>/project-<env>/<portfolio>/group_vars/<service>/variables.yml</code>

More inventories may be added as needed, in the same manner.

Follow-up plan

Embed the previous command in a script to be called in the systemd service with `ExecStartPre` (add `TimeoutSec=30m` in accordingly) so it configures the service before starting it.

The script is able to gather the environment from the machine metadata and configure our service.